

GUIA DE LABORATORIO 05

CURSO DE PROGRAMACIÓN APLICADA II

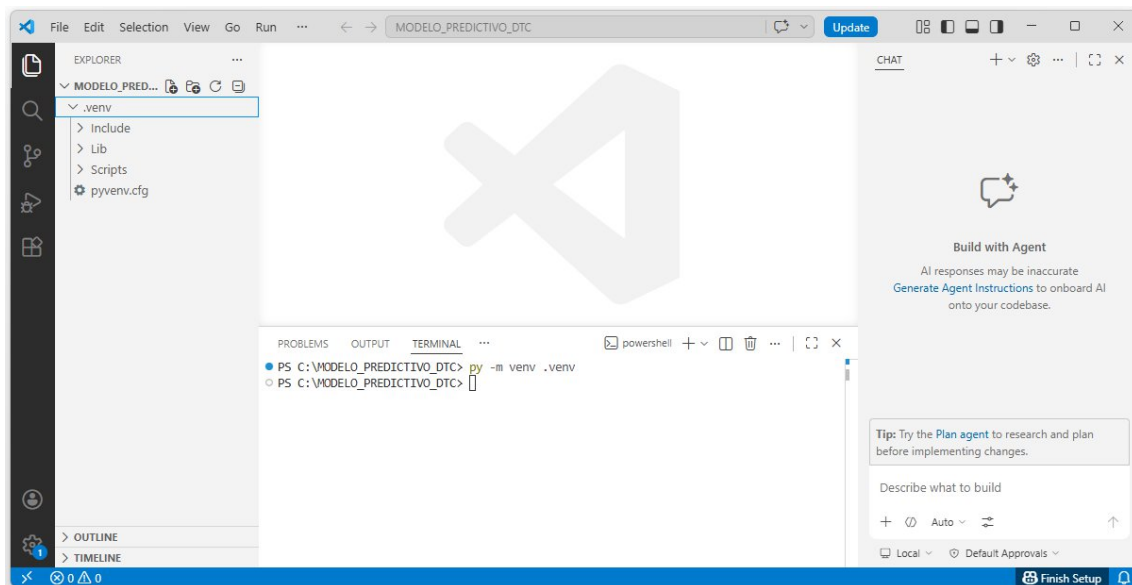
**CLASIFICACIÓN DE TUMORES CEREBRALES EN IMÁGENES
MRI MEDIANTE FLUTTER, FLASK Y CNN**

DR. IVAN CARLO PETRLIK AZABACHE

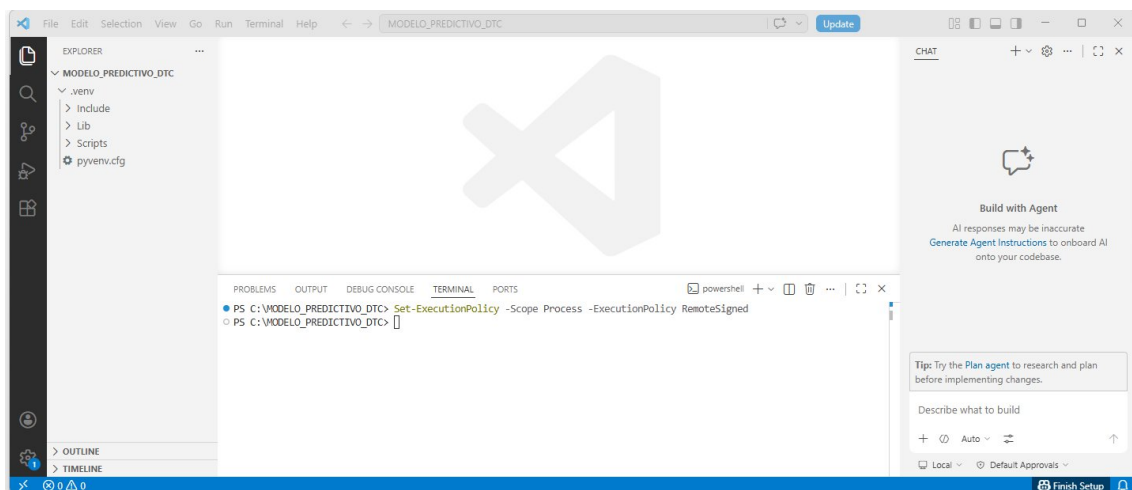
LIMA - PERU

2026

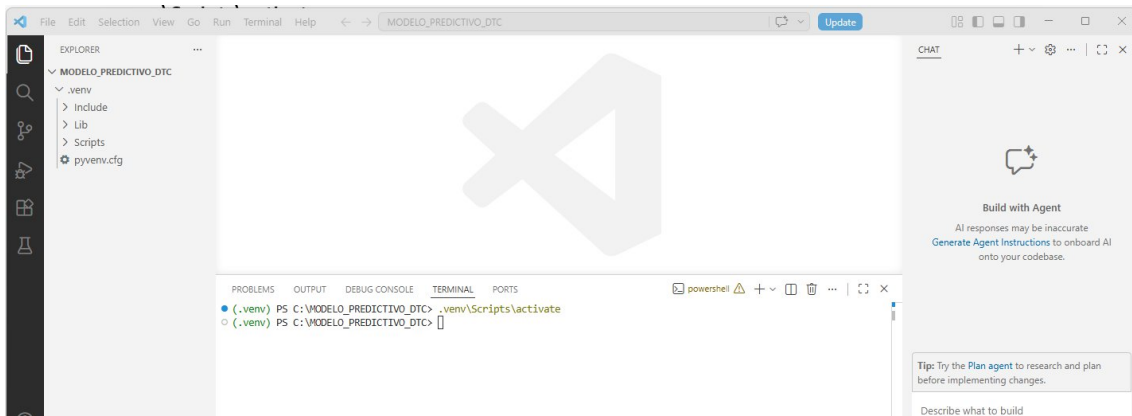
py -m venv .venv



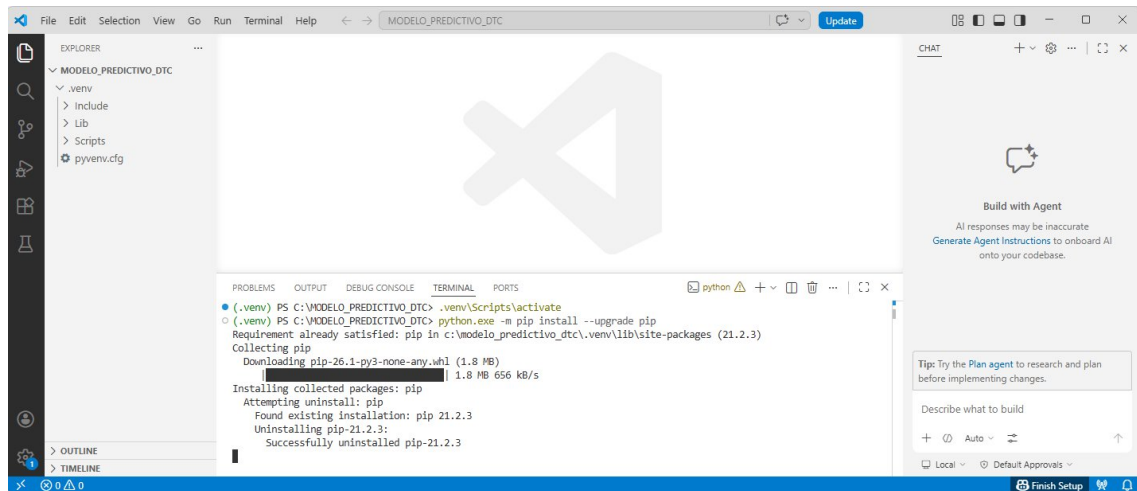
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned



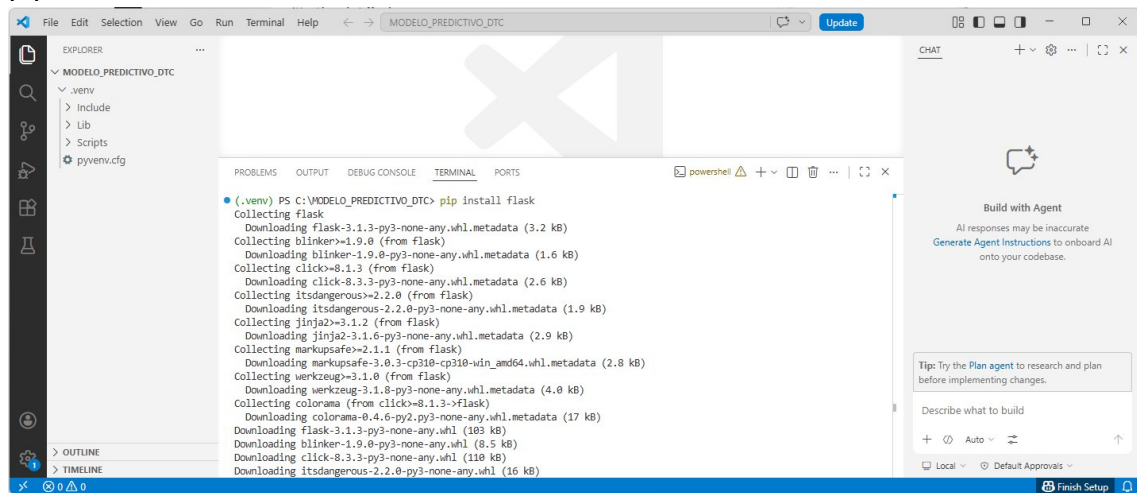
.venv\Scripts\activate



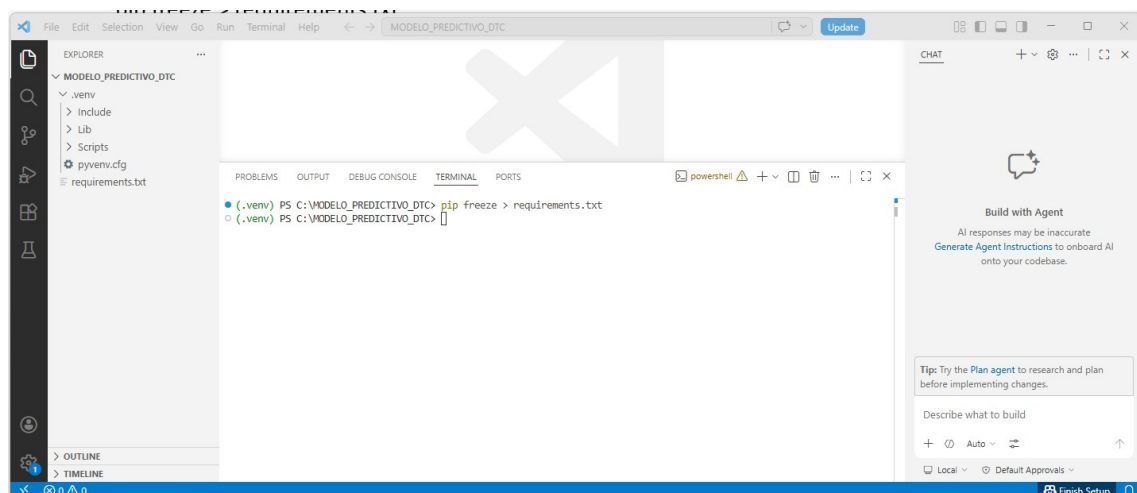
python.exe -m pip install --upgrade pip



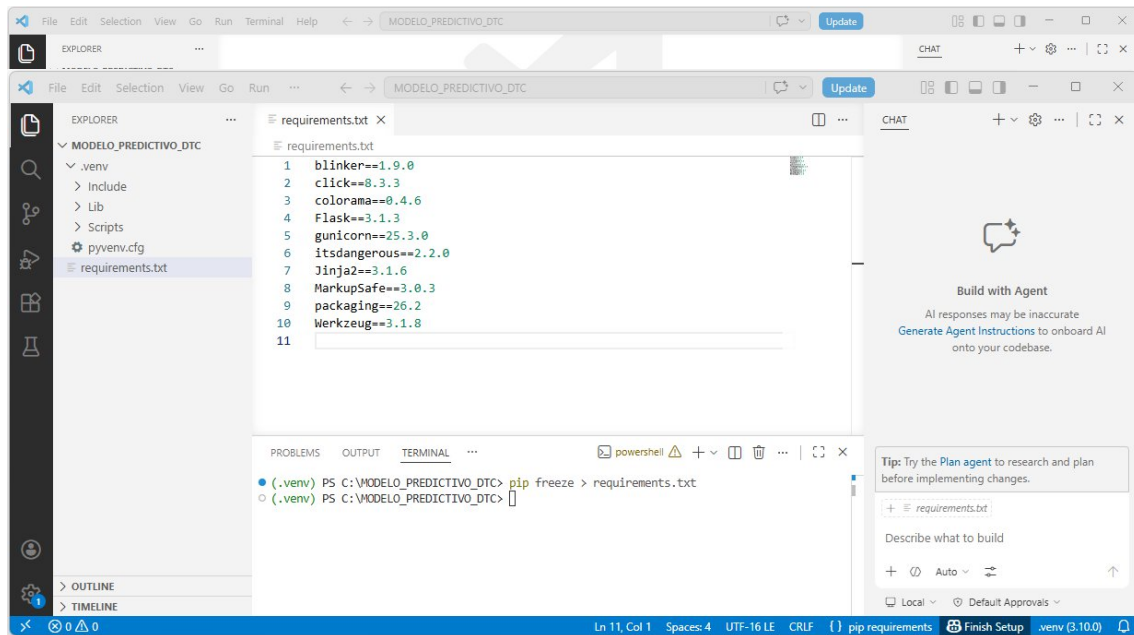
pip install flask



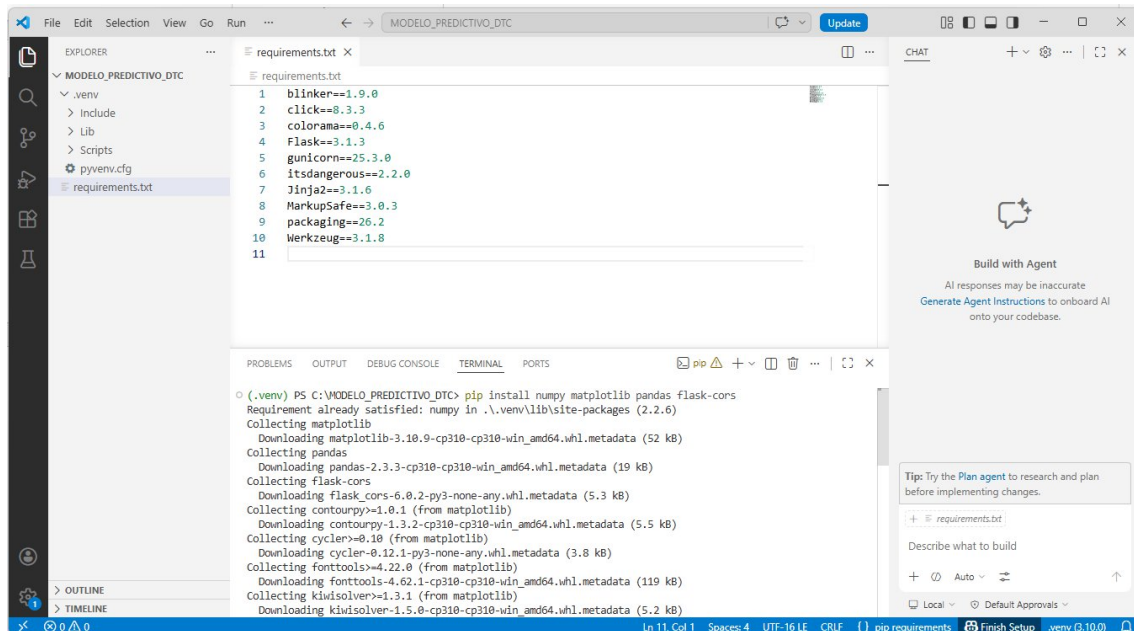
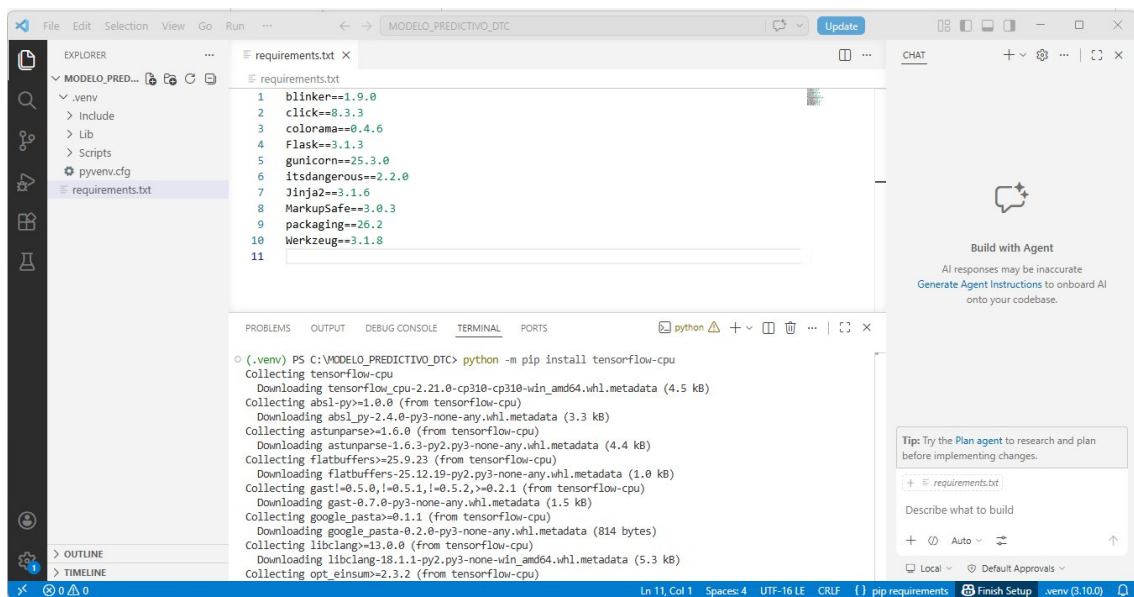
pip freeze > requirements.txt



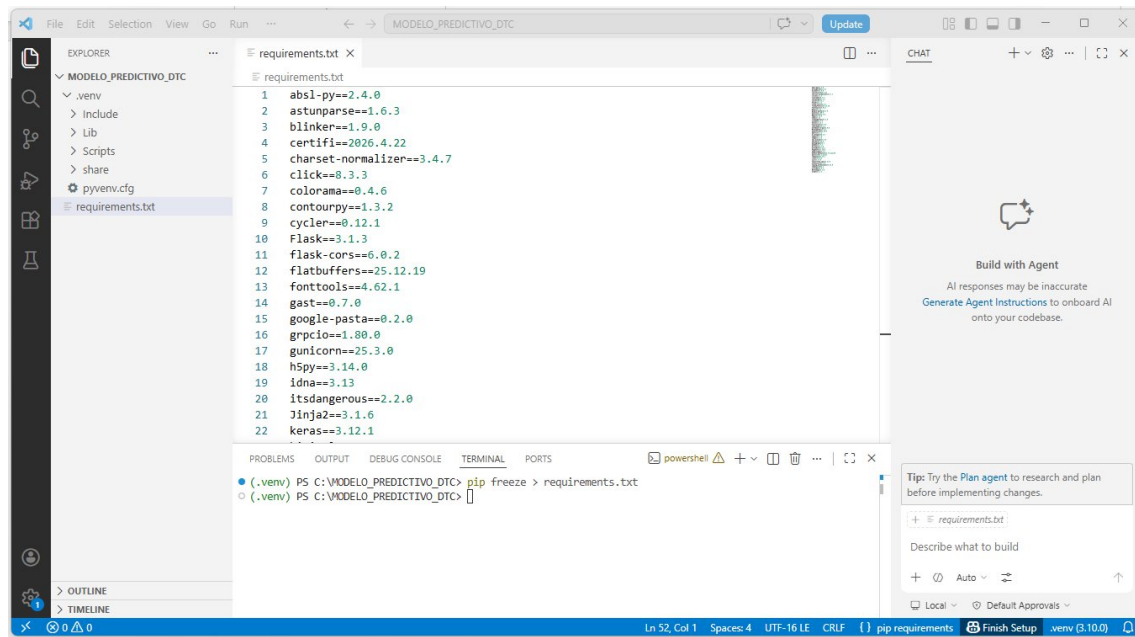
pip install gunicorn



python -m pip install tensorflow-cpu



pip freeze > requirements.txt



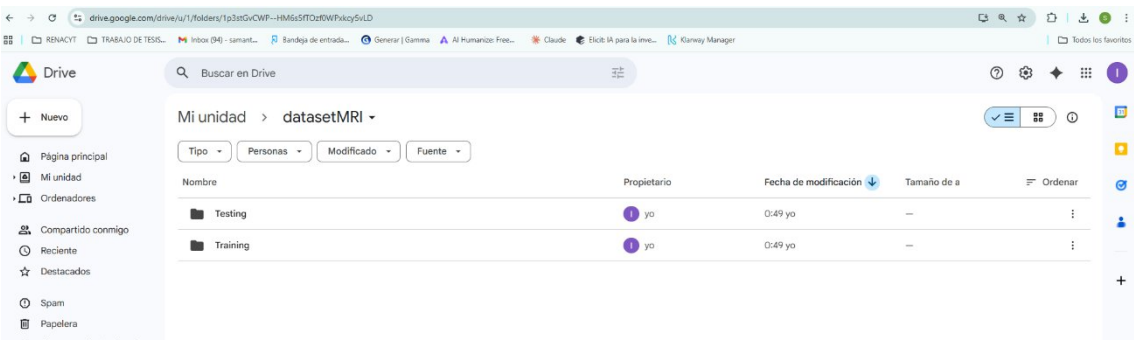
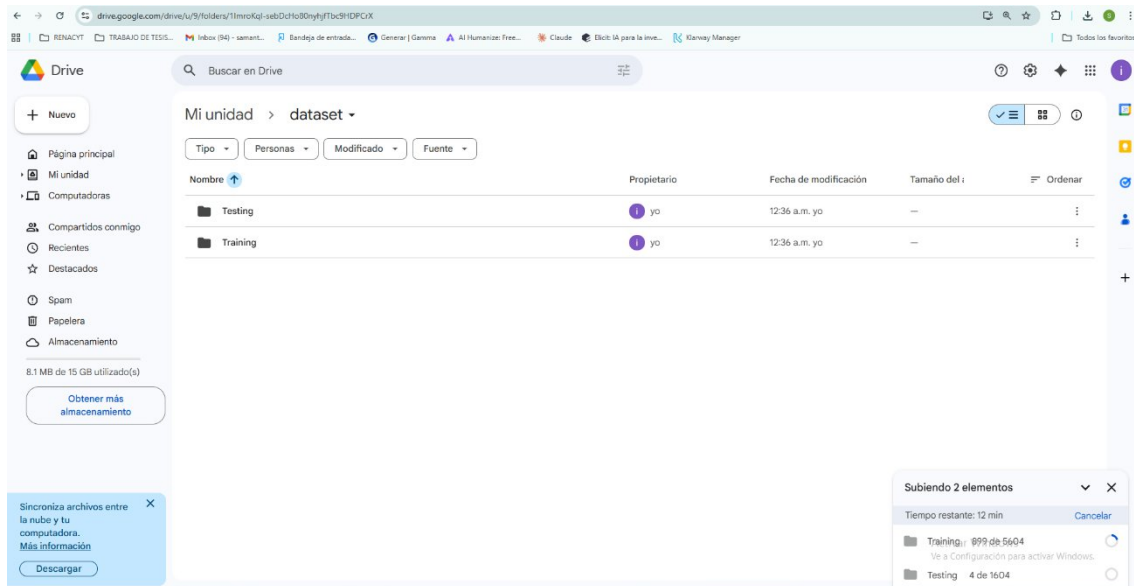
A continuación, descargaremos el dataset de imágenes MRI desde la plataforma **Kaggle**, el cual será utilizado para el entrenamiento y validación del modelo de inteligencia artificial.

Enlace del dataset:

<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset?resource=download>



Subimos las carpetas al Google drive :



Ahora ejecutando en el google colab

```

# CLASIFICACIÓN DE TUMORES CEREBRALES CON CNN DESDE GOOGLE DRIVE
# GOOGLE COLAB

# 1. Importación de librerías
import os
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from google.colab import drive
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix

# 2. Montar Google Drive
drive.mount('/content/drive')

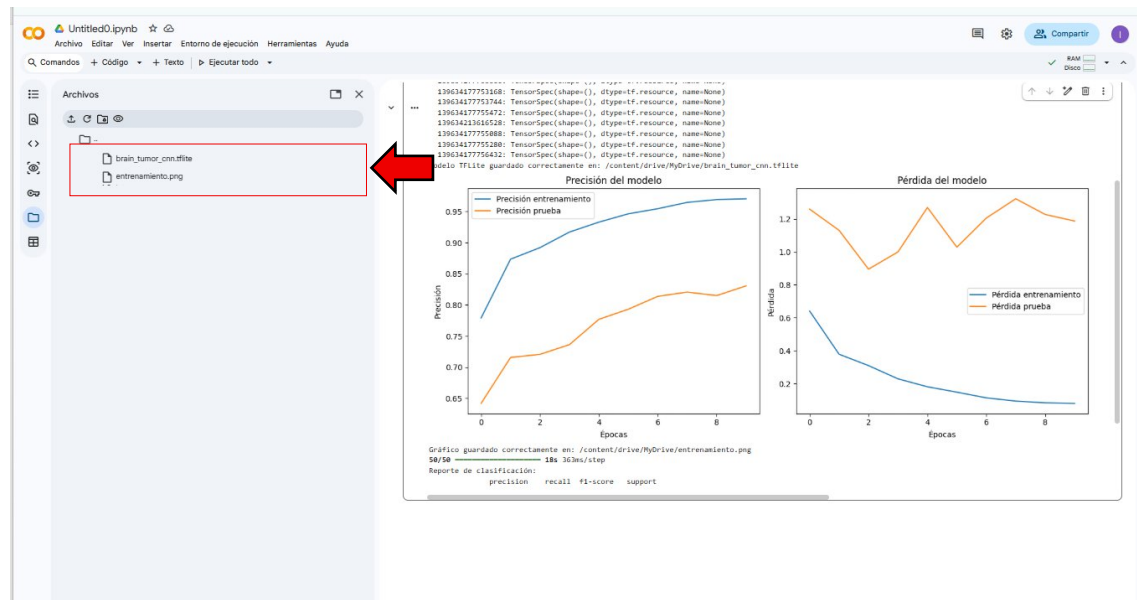
# 3. Ruta del dataset en Google Drive

# Cambiar esta ruta si tu carpeta tiene otro nombre o está en otra ubicación
base_path = "/content/drive/MyDrive/dataset/MI"

train_dir = os.path.join(base_path, "Training")
test_dir = os.path.join(base_path, "Testing")

print("Ruta de entrenamiento:", train_dir)
print("Ruta de prueba:", test_dir)
print("¿Existe carpeta 'training'?", os.path.exists(train_dir))
print("¿Existe carpeta 'testing'?", os.path.exists(test_dir))

```



```

# =====
# CLASIFICACIÓN DE TUMORES CEREBRALES CON CNN DESDE GOOGLE DRIVE
# GOOGLE COLAB
# =====

```

```

# 1. Importación de librerías
import os

```

```

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from google.colab import drive
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix

# =====
# 2. Montar Google Drive
# =====

drive.mount('/content/drive')

# =====
# 3. Ruta del dataset en Google Drive
# =====

# Cambiar esta ruta si tu carpeta tiene otro nombre o está en otra ubicación
base_path = "/content/drive/MyDrive/datasetMRI"

train_dir = os.path.join(base_path, "Training")
test_dir = os.path.join(base_path, "Testing")

print("Ruta de entrenamiento:", train_dir)
print("Ruta de prueba:", test_dir)
print("¿Existe carpeta Training?:", os.path.exists(train_dir))
print("¿Existe carpeta Testing?:", os.path.exists(test_dir))

# =====
# 4. Validación de rutas
# =====

if not os.path.exists(train_dir):
    raise FileNotFoundError("No se encontró la carpeta Training. Verifica la ruta en Google Drive.")

if not os.path.exists(test_dir):
    raise FileNotFoundError("No se encontró la carpeta Testing. Verifica la ruta en Google Drive.")

# =====
# 5. Generadores de imágenes
# =====

train_datagen = ImageDataGenerator(
    rescale=1./255
)

test_datagen = ImageDataGenerator(
    rescale=1./255
)

train_gen = train_datagen.flow_from_directory(
    train_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical'
)

test_gen = test_datagen.flow_from_directory(
    test_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)

# =====
# 6. Verificación de clases
# =====

print("Clases detectadas:", train_gen.class_indices)

num_clases = len(train_gen.class_indices)

print("Número de clases:", num_clases)

# =====
# 7. Construcción del modelo CNN
# =====

model = models.Sequential([
    layers.Conv2D(
        32,
        (3, 3),
        activation='relu',
        input_shape=(128, 128, 3)
    ),
    layers.MaxPooling2D(),

    layers.Conv2D(
        64,
        (3, 3),
        activation='relu'
    ),
    layers.MaxPooling2D(),

    layers.Conv2D(
        128,
        (3, 3),
        activation='relu'
    )
])

```



```

    ),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(
        128,
        activation='relu'
    ),

    layers.Dropout(0.5),

    layers.Dense(
        num_classes,
        activation='softmax'
    )
})

model.summary()

# =====
# 8. Compilación del modelo
# =====

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# =====
# 9. Entrenamiento del modelo
# =====

history = model.fit(
    train_gen,
    validation_data=test_gen,
    epochs=10
)

# =====
# 10. Conversión del modelo a TensorFlow Lite
# =====

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

ruta_tflite = "/content/drive/MyDrive/brain_tumor_cnn.tflite"

with open(ruta_tflite, 'wb') as f:
    f.write(tflite_model)

print("Modelo TFLite guardado correctamente en:", ruta_tflite)

# =====
# 11. Gráficos de precisión y pérdida
# =====

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Precisión entrenamiento')
plt.plot(history.history['val_accuracy'], label='Precisión prueba')
plt.title("Precisión del modelo")
plt.xlabel("Épocas")
plt.ylabel("Precisión")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Pérdida entrenamiento')
plt.plot(history.history['val_loss'], label='Pérdida prueba')
plt.title("Pérdida del modelo")
plt.xlabel("Épocas")
plt.ylabel("Pérdida")
plt.legend()

plt.tight_layout()

ruta_grafico = "/content/drive/MyDrive/entrenamiento.png"
plt.savefig(ruta_grafico)
plt.show()

print("Gráfico guardado correctamente en:", ruta_grafico)

# =====
# 12. Evaluación del modelo
# =====

test_gen.reset()

predictions = model.predict(test_gen)

predicted_classes = np.argmax(predictions, axis=1)

true_classes = test_gen.classes

class_labels = list(test_gen.class_indices.keys())

# =====
# 13. Reporte de clasificación
# =====

```

```

report = classification_report(
    true_classes,
    predicted_classes,
    target_names=class_labels
)

print("Reporte de clasificación:")
print(report)

# =====
# 14. Matriz de confusión
# =====

matrix = confusion_matrix(
    true_classes,
    predicted_classes
)

print("Matriz de confusión:")
print(matrix)

```

A través del siguiente enlace de Google Colab, se podrá acceder al código utilizado para el entrenamiento y generación del modelo de inteligencia artificial:

<https://colab.research.google.com/drive/1sScbGhKmARWuZNXDf77NPptH1aXMoRtZ?usp=sharing>

Asimismo, se deberá descargar el modelo de IA generado en formato **TFLite** desde el siguiente enlace:

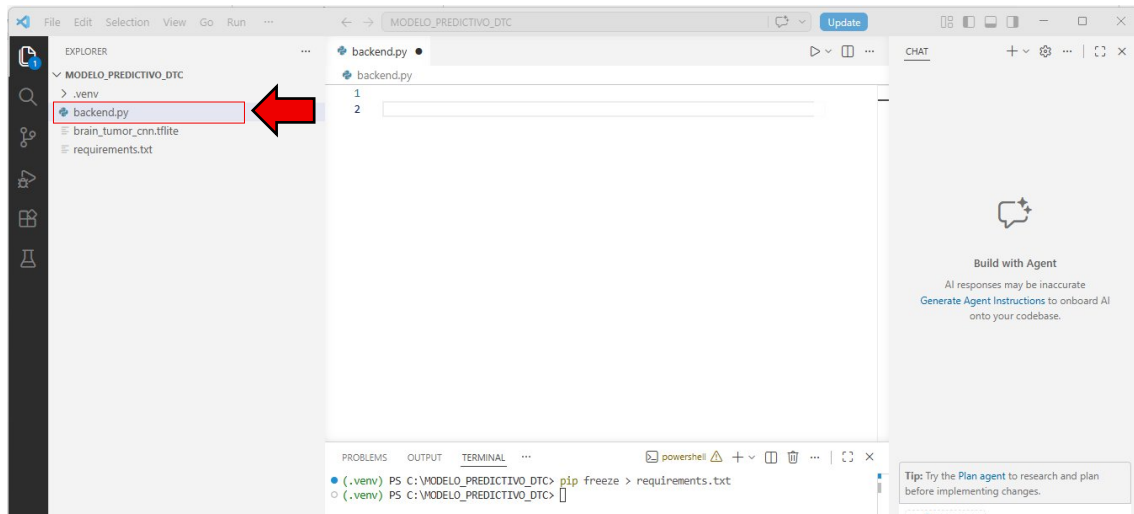
https://drive.google.com/file/d/1ocZQoUYXR01sgIJVpEf9XlsG0Bg3j_-C/view?usp=sharing

Una vez descargado el archivo del modelo en formato. tflite, este deberá colocarse dentro del proyecto desarrollado en **Flask**, preferentemente en la carpeta raíz del proyecto.

Ahora este modelo en formato tflite se tiene que colocar dentro del proyecto en flask



Ahora creando el archivo de nombre backend.py



Ahora dentro de este archivo backend.py se tiene que colocar el siguiente código:

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing import image
import os
from werkzeug.utils import secure_filename
import matplotlib

matplotlib.use('Agg')
import matplotlib.pyplot as plt

app = Flask(__name__)
CORS(app)

interpreter = tf.lite.Interpreter(model_path='brain_tumor_cnn.tflite')
interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

class_names = ['glioma', 'meningioma', 'notumor', 'pituitary']

UPLOAD_FOLDER = os.path.join('static', 'uploads')
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

def predict_with_tflite(img_array):
    img_array = img_array.astype(np.float32) / 255.0

    interpreter.set_tensor(input_details[0]['index'], img_array)
    interpreter.invoke()

    output_data = interpreter.get_tensor(output_details[0]['index'])

    return output_data[0]

@app.route('/api/clasificar', methods=['POST'])
def clasificar_api():
    file = request.files.get('image')

    if not file:
        return jsonify({'error': 'No se envió imagen'}), 400

    filename = secure_filename(file.filename)
    filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
    file.save(filepath)

    img = image.load_img(filepath, target_size=(128, 128))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)

    prediction = predict_with_tflite(img_array)
    predicted_class = class_names[np.argmax(prediction)]

    probabilities = {
        class_names[i]: float(f'{prob:.4f}')
        for i, prob in enumerate(prediction)
    }

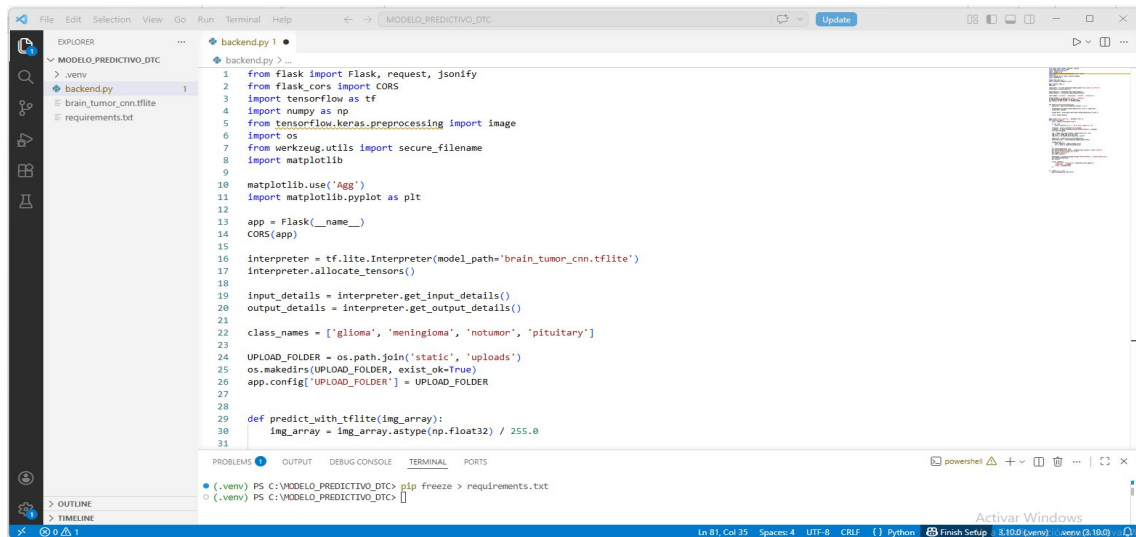
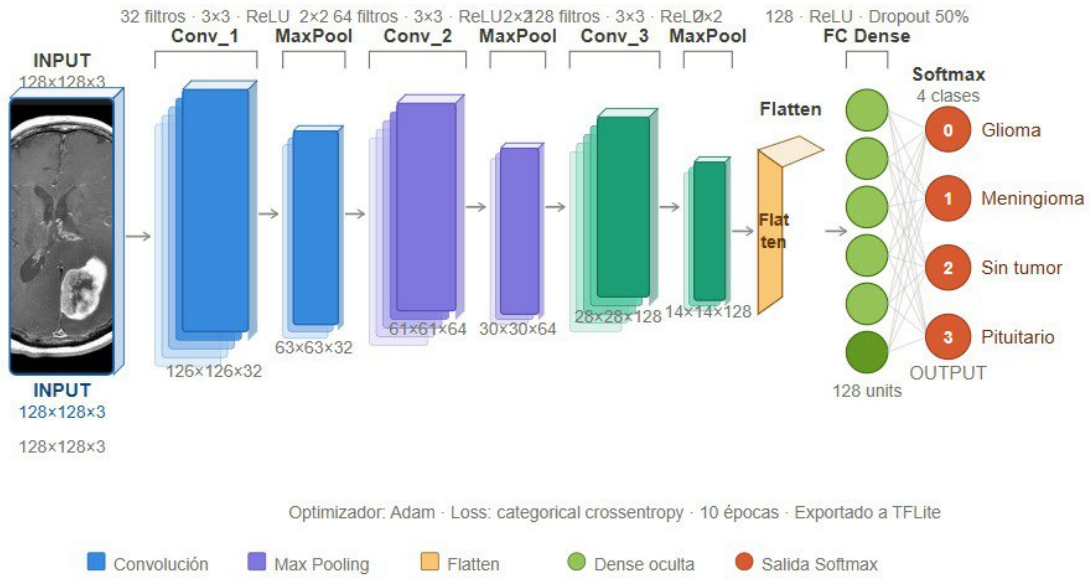
    plt.figure(figsize=(6, 4))
    plt.bar(probabilities.keys(), probabilities.values(), color='skyblue')
    plt.title('Probabilidades por clase')
    plt.ylabel('Confianza')
    plt.tight_layout()

    graph_path = os.path.join(app.config['UPLOAD_FOLDER'], 'probabilidades.png')
    plt.savefig(graph_path)
    plt.close()

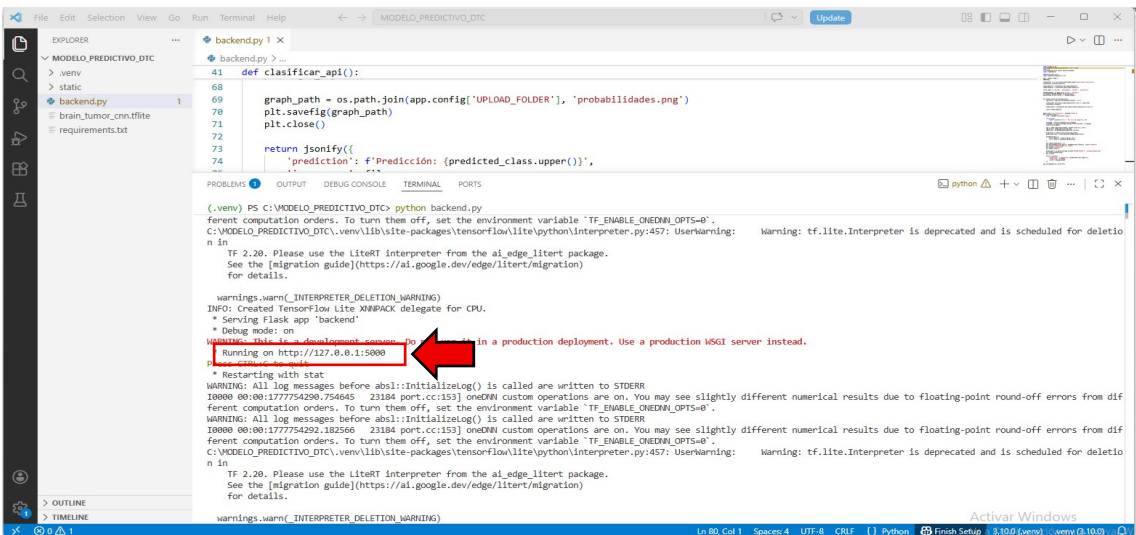
    return jsonify({
        'prediction': f'Predicción: {predicted_class.upper()}',
        'image_name': filename,
        'probs': probabilities
    })

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

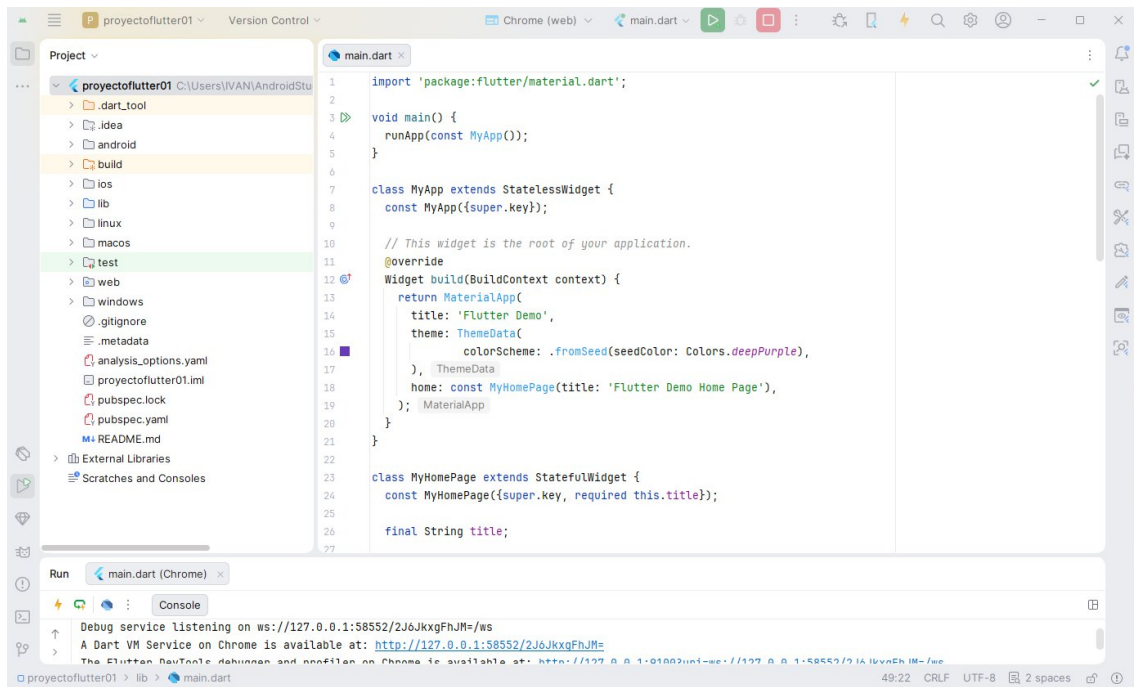
ARQUITECTURA



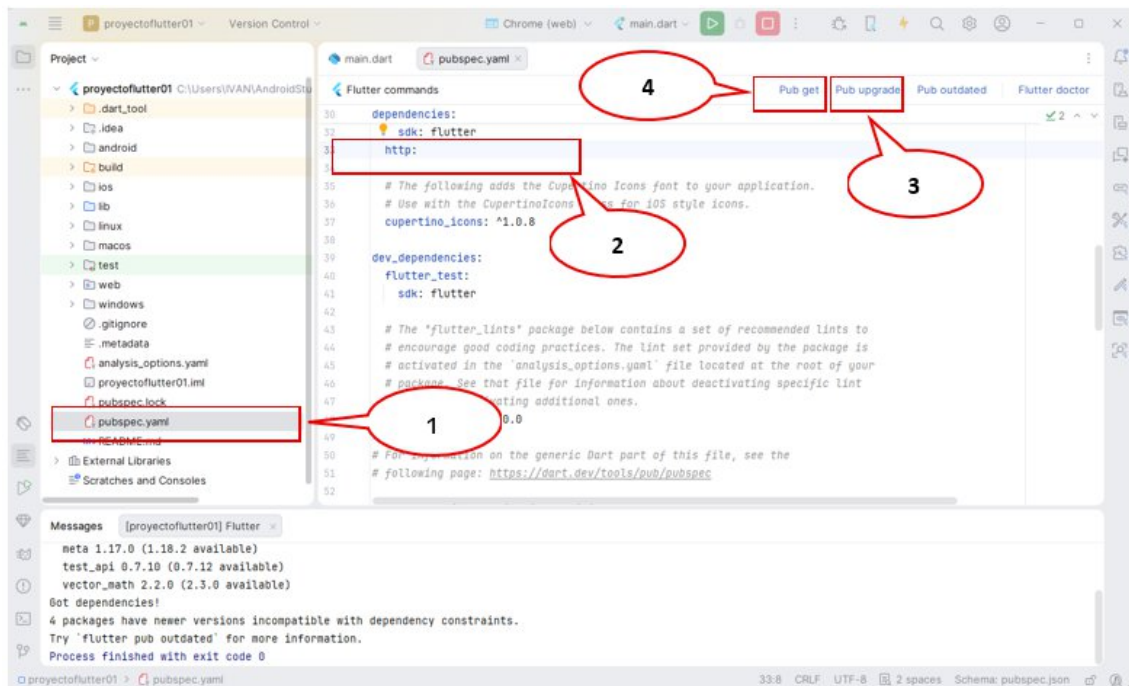
Ejecutando la aplicación



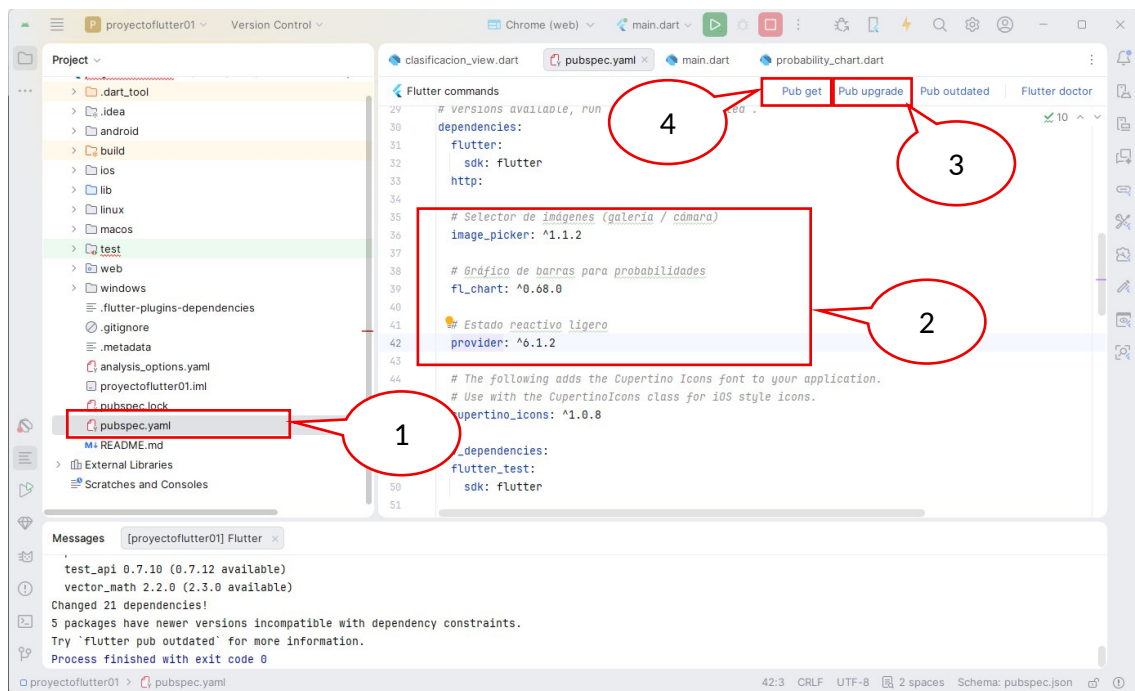
A continuación, vamos a desarrollar la aplicación móvil en flutter



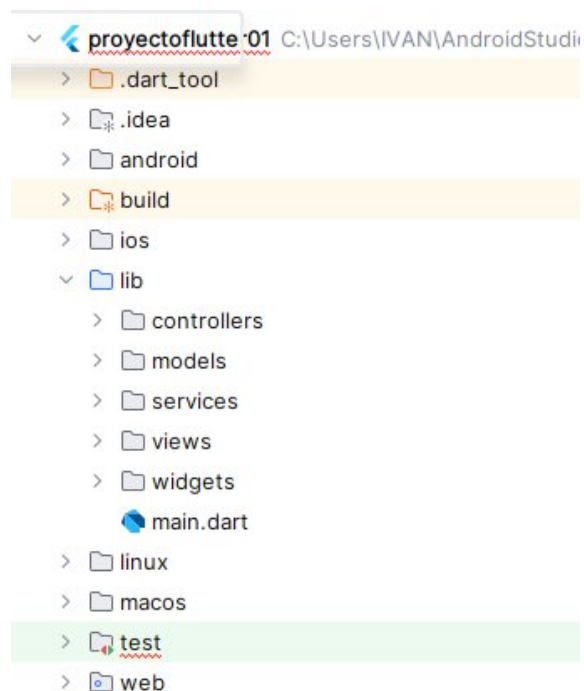
A continuación, vamos a configurar el acceso al protocolo http



Aquí también se configuran algunas librerías adicionales que serán utilizadas durante el desarrollo del código:



Ahora, dentro de la carpeta `lib`, se deberán crear las subcarpetas y los archivos de código en Dart, organizados bajo el patrón de arquitectura MVC, de la siguiente manera:



El proyecto Flutter completo está listo. Aquí un resumen de lo que contiene:

Estructura MVC

Capa	Archivo	Responsabilidad
Model	<code>models/clasificacion_result.dart</code>	Mapea el JSON de Flask,

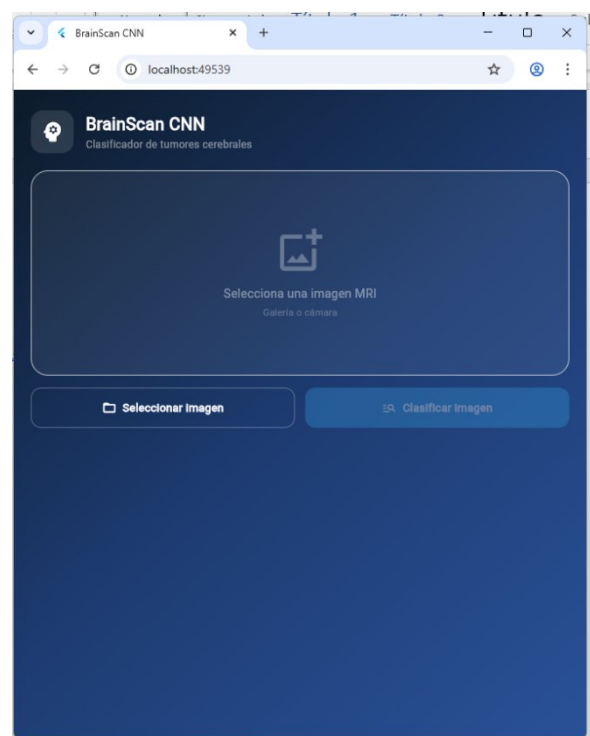
		incluye helpers de formato y color por clase
Service	services/api_service.dart	POST multipart al endpoint /api/clasificar, manejo de errores tipados
Controller	controllers/clasificacion_controller.dart	Estado reactivo (inicial/cargando/éxito/error), orquesta picker → API
View	views/clasificacion_view.dart	UI completa, no tiene lógica de negocio
Widgets	widgets/probability_chart.dart result_card.dart	+ Gráfico de barras con fl_chart + tarjeta de resultado con descripción clínica

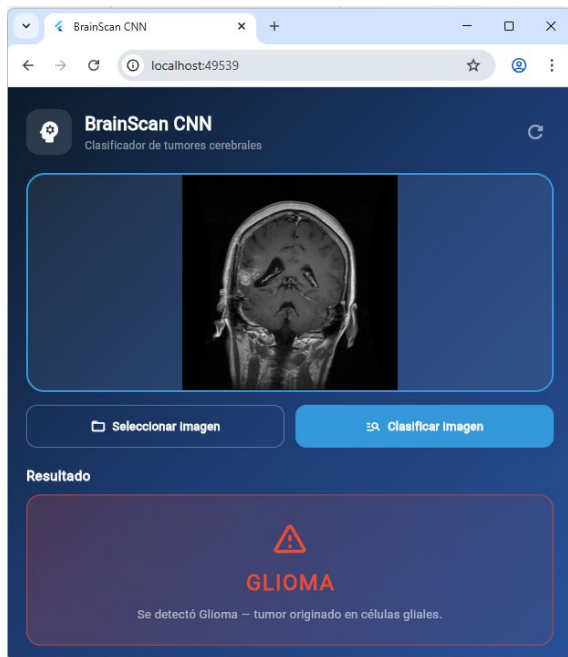
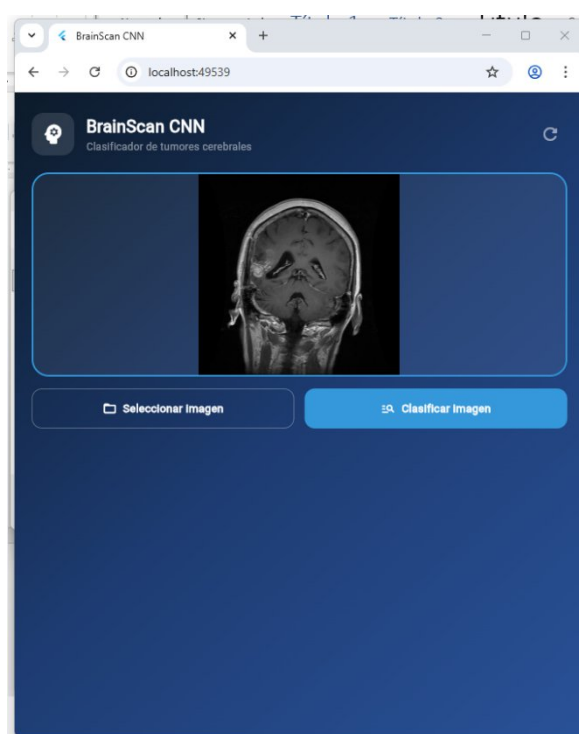
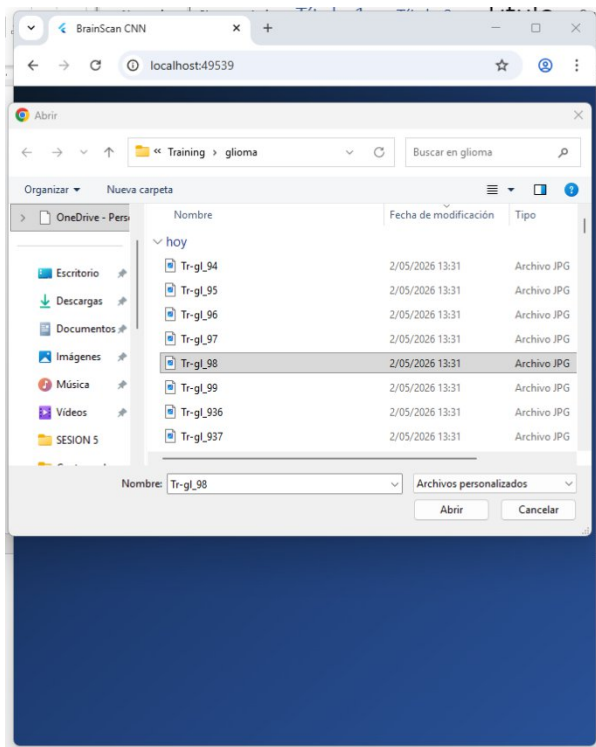
A continuación, compartiré el conjunto de carpetas y el código en Dart correspondiente, para que puedan copiarlo y pegarlo dentro de la carpeta **lib** del proyecto desarrollado en Flutter.

Enlace de acceso:

https://drive.google.com/drive/folders/1N_u523P3HACJCpt9WLRMFhZ6Rh2kSnh9?usp=sharing

Ahora procedemos a ejecutarlo





EJERCICIO PROPUESTO

En base al ejercicio desarrollado, se deberá desplegar la aplicación del proyecto en Flask en el servicio Render. Asimismo, la aplicación móvil deberá compilarse y generar su respectivo archivo APK para su instalación y prueba en dispositivos Android.